

Software Optimizations Become Simple with Top-Down Analysis Methodology on Intel® Microarchitecture, Code Name Skylake

Ahmad Yasin, Senior CPU Architect, Intel Corporation

ARCS002

Agenda

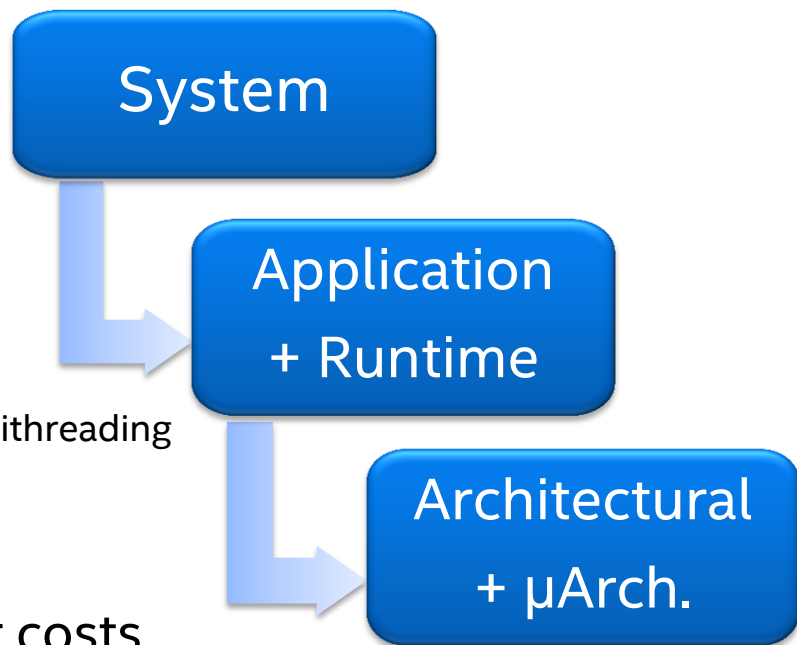
- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

Agenda

- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

Background

- Performance Analysis Process
 - System Level
 - Examples: Disk, Network, Graphics
 - Application Level
 - Examples: Algorithm complexity, Parallelization/Multithreading
 - Architectural and microarchitectural Levels
 - Examples: vectorize key loops, reduce cache misses
- **Goal: detect bottlenecks** & estimate their costs
 - 2nd goal: locate issues (event-based sampling)
 - Not-a-goal: quantify speedup
 - Assumption: CPU Bound workload

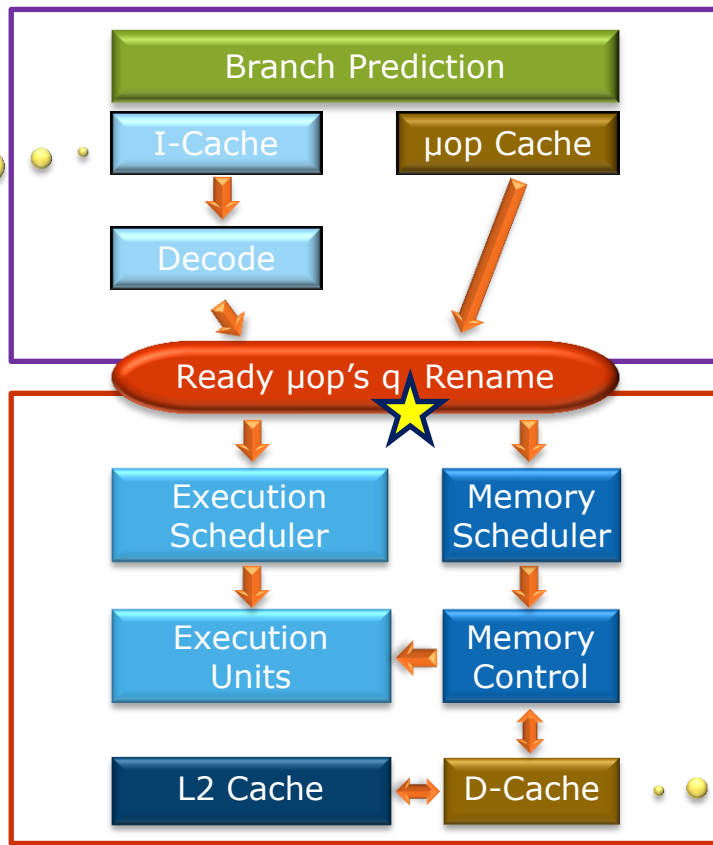


Preface

- Problem: Performance Optimization Is Difficult
 - Sophisticated microarchitectures
 - Application/workload diversity
 - Unmanageable profiling data
 - Tougher constraints (Time To Market, Resources, Priorities)
- Solution: **Top-down Microarchitecture Analysis Method (TMAM)**
 - Identifies **true** μ -arch bottlenecks in a simple, **structured** hierarchical process
 - **Analysis is made easier** for users who may lack hardware expertise
 - Simplified hierarchy avoids the μ -arch high-learning curve
 - Learn method once; use it on all Intel Architecture (IA) based processors
 - TMAM full hierarchy is supported on Core™ and Xeon™ processors
 - TMAM level-1 is available with recent generations of Atom™ and Xeon Phi™ processors

Skylake Core

+ an instruction cache miss in next function g()



Front-end
of processor
pipeline

Back-end
of processor
pipeline

A data cache
miss in current
function, say f()

TMAM is designed to help developer to focus on areas that matter

Challenges

Traditional Methods

- Naïve approach

$$\text{Stall_Cycles} = \sum \text{Penalty}_i * \text{MissEvent}_i$$

- Unsuitable for modern out-of-order cores due to (Gaps):
 - 1) *Stalls Overlap*
 - 2) Speculative Execution
 - 3) Workload-dependent penalties
 - 4) Predefined set of miss-events
 - 5) Superscalar inaccuracy

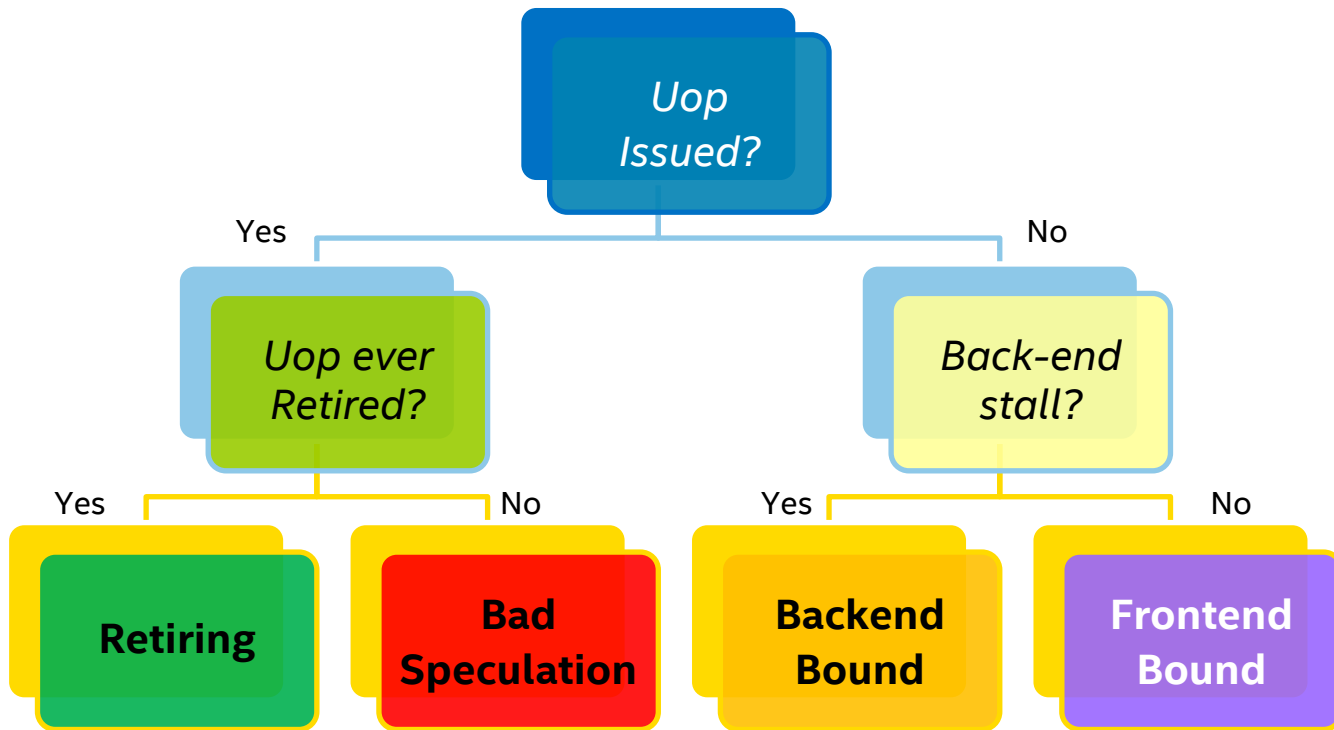
Top Down Analysis

- A hierarchy
 - Top-down designated events at appropriate pipeline stages
 - “Hierarchical safety property”
- Addressing Gaps
 - **Bad Speculation at the top**
 - Generic top-down orients events that
 - count when matters; e.g. stalls vs cycles
 - count where matters; e.g. single point of division
 - count at finer-grain; e.g. issue-pipeline width.

Agenda

- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

Top Level Breakdown



Uop := micro-operation. Each x86 instruction is decoded into uop(s)

Uop Issue := last front-end stage where a uop is ready to acquire back-end resources

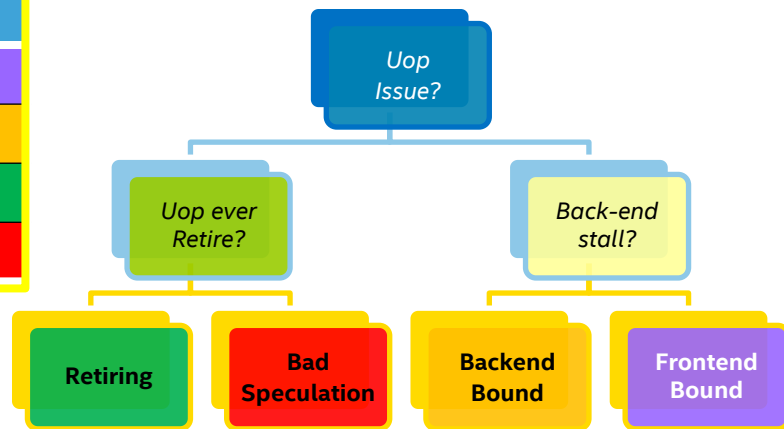
Back-end stall := Any backend resource fills up which blocks issue of new uops

Top Level Breakdown Illustration

Cycle	1	2	3	4	...	x
Back-end Stall	0	0	1	0		0
Issue Slot 0	-	v	-	v		v
Issue Slot 1	-	v	-	v		v
Issue Slot 2	-	-	-	v		v
Issue Slot 3	-	-	-	v		-
Frontend Bound	4	2		0		1
Backend Bound			4	0		0
Retiring		2		1		2
Bad Speculation				3		1

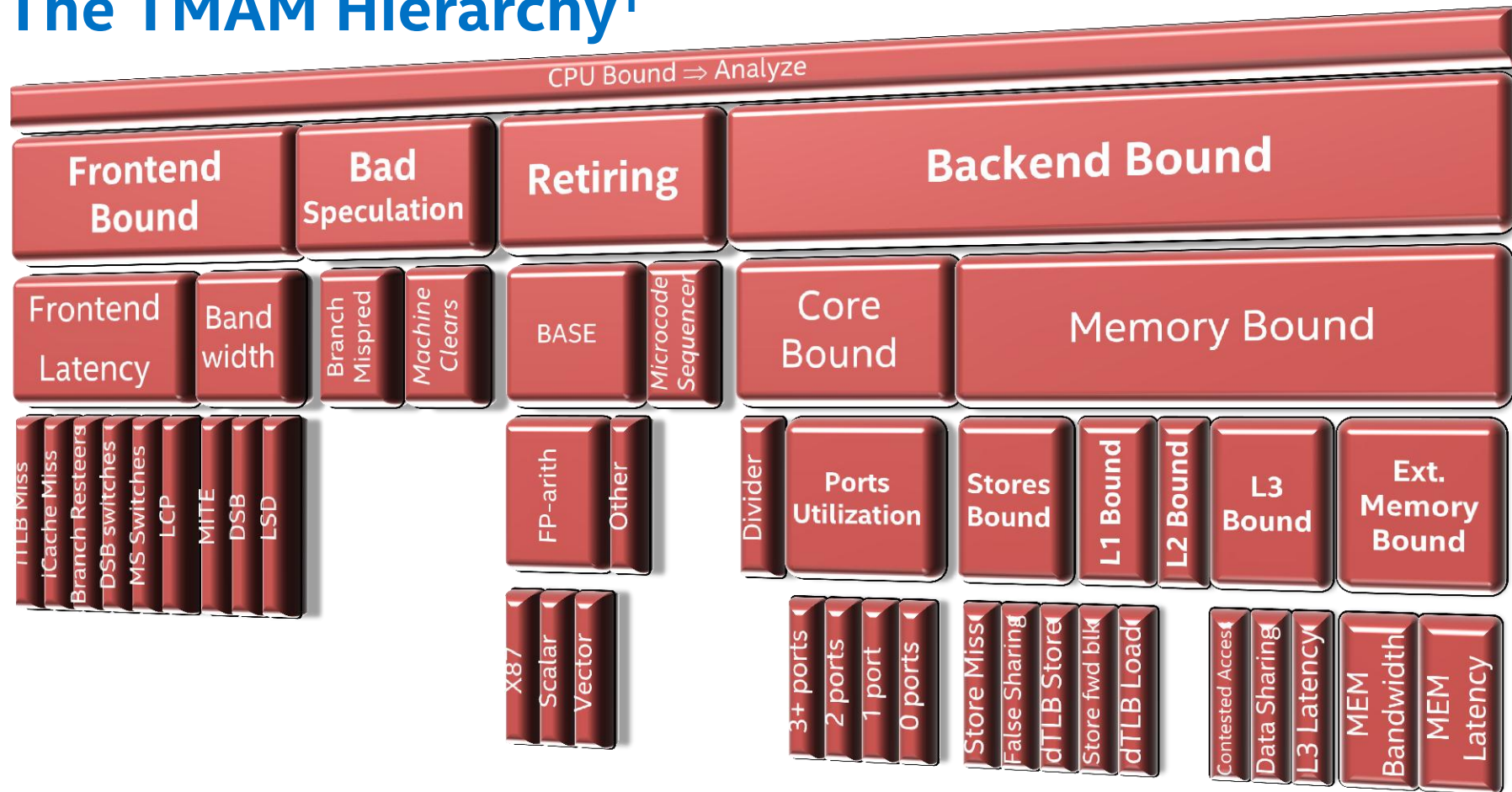
Issue Slots := number of uops the front-end pipeline can issue to the back-end in a cycle.

Bubble := An un-utilized Slot.



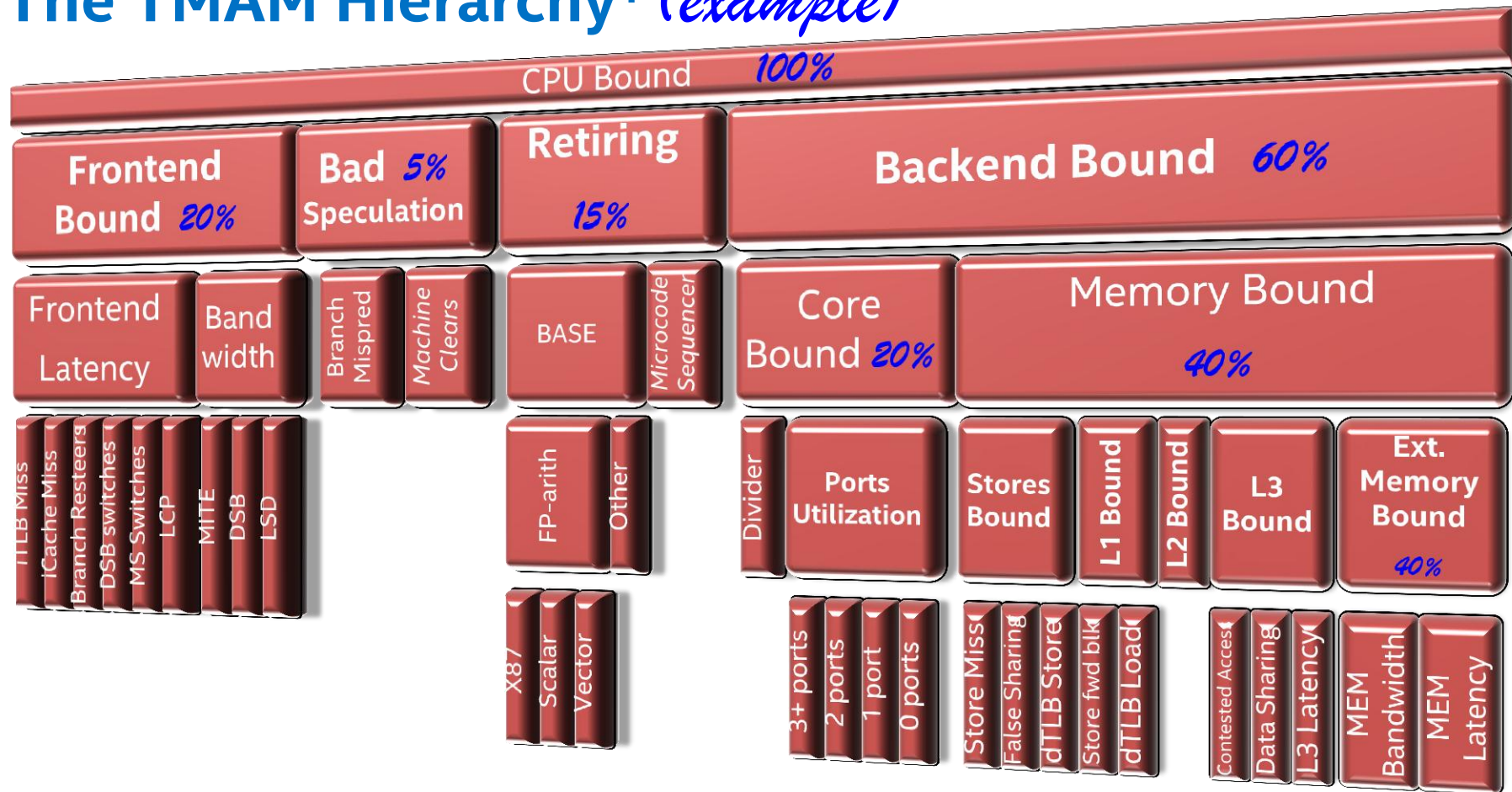
Classify Each Issue Slot Into 1 of 4 Categories

The TMAM Hierarchy¹



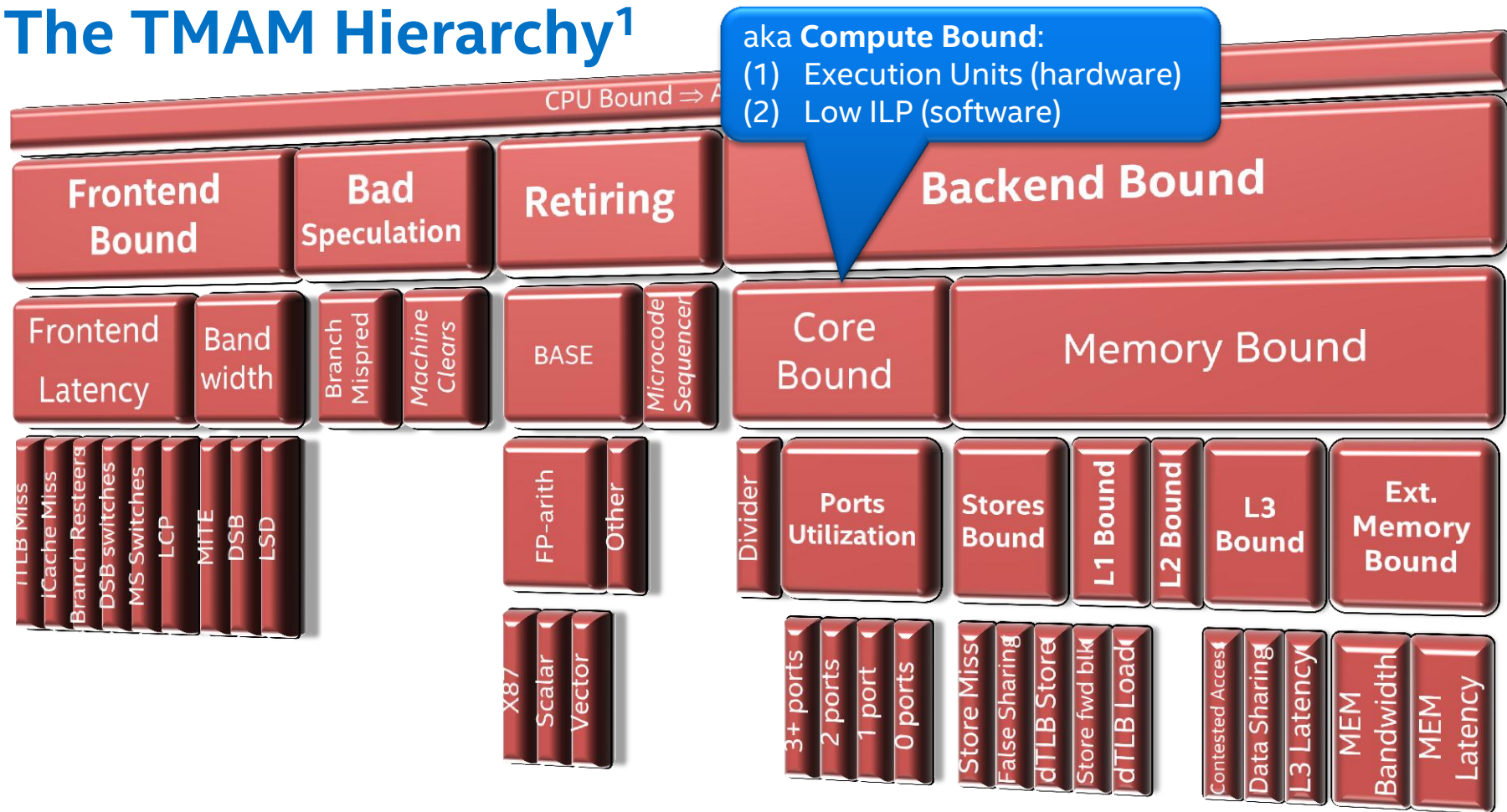
[1] A. Yasin, "A Top-Down Method for Performance Analysis and Counters Architecture", ISPASS 2014

The TMAM Hierarchy¹ (example)



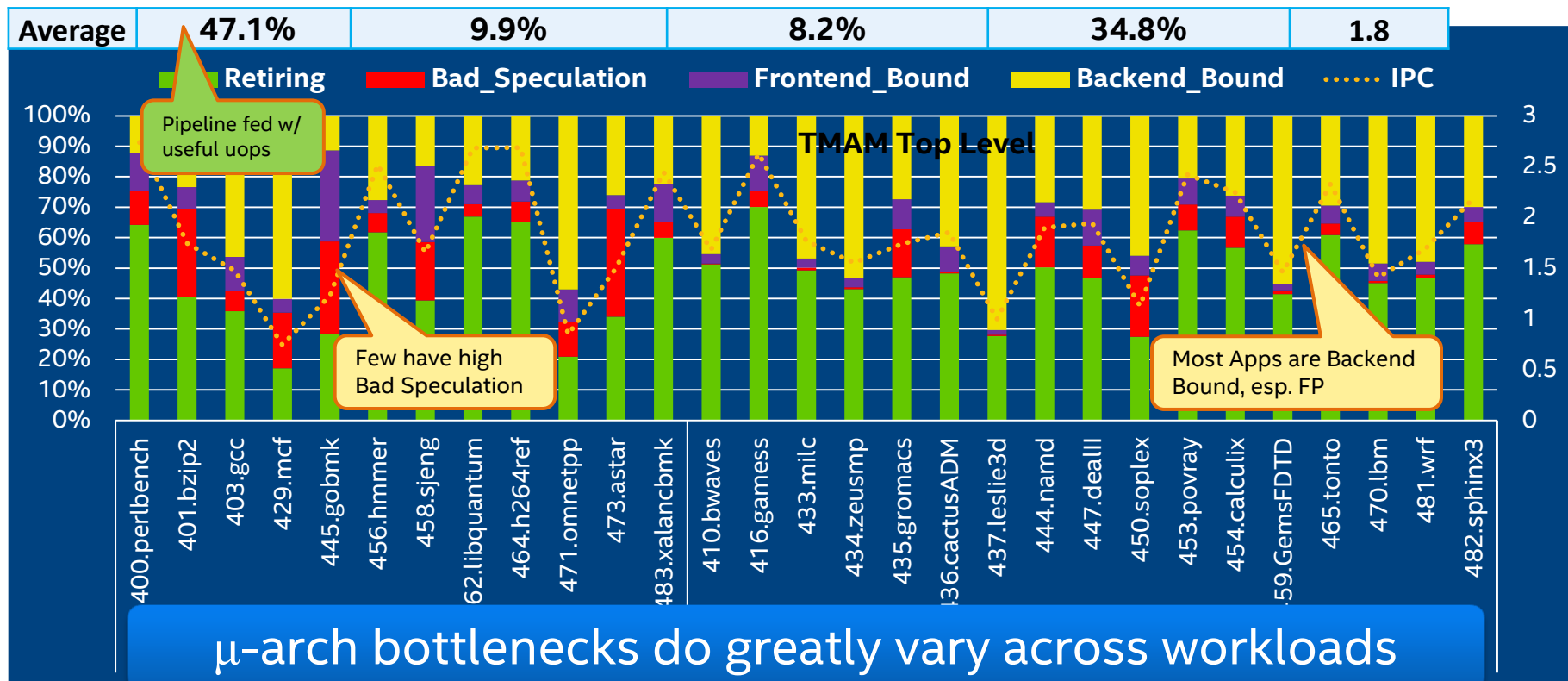
[1] A. Yasin, "A Top-Down Method for Performance Analysis and Counters Architecture", ISPASS 2014

The TMAM Hierarchy¹



[1] A. Yasin, "A Top-Down Method for Performance Analysis and Counters Architecture", ISPASS 2014

Top Level for SPEC CPU2006

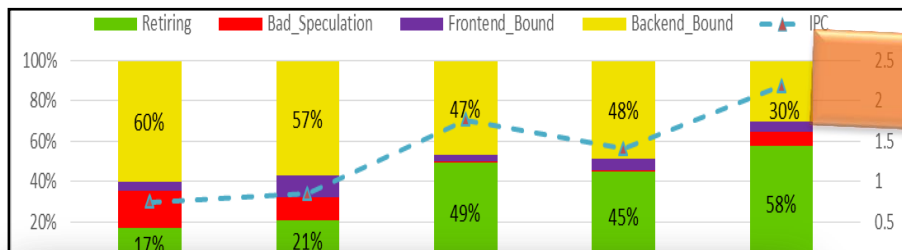


μ -arch bottlenecks do greatly vary across workloads

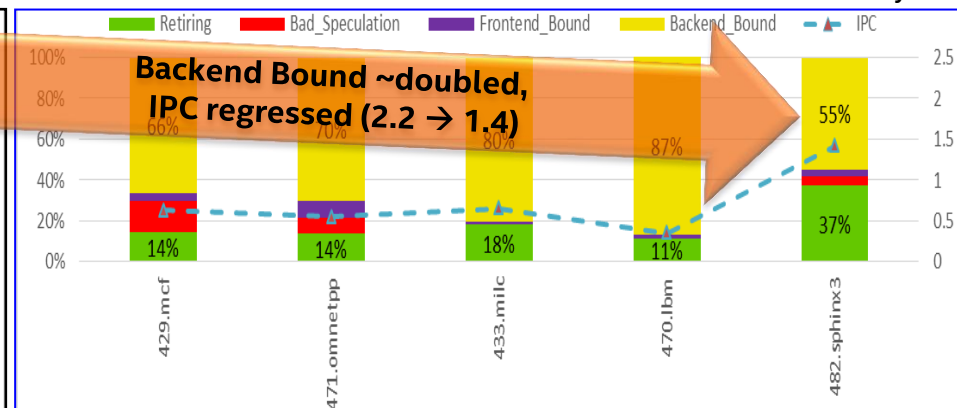
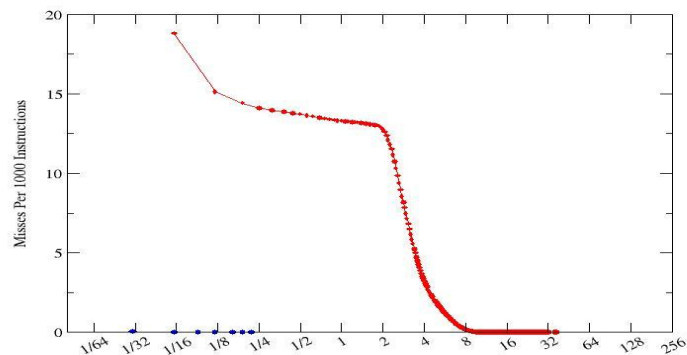
SPEC CPU2006 v1.2, rate 1-copy, Intel Compiler 14 targeting AVX2, Skylake @ 3 GHz

Memory Bound (1-core vs. 4-cores)

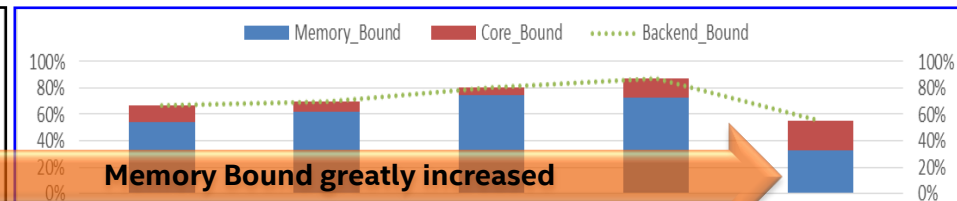
IPC := Instructions Per Cycle



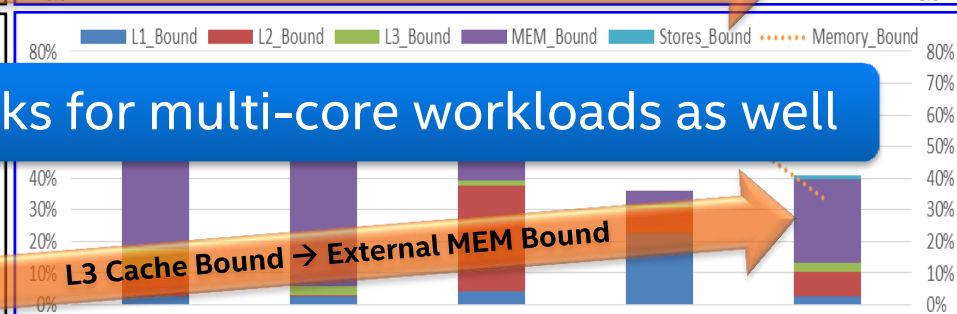
sphinx3.an4 - Working Set Analysis



Backend Bound ~doubled,
IPC regressed (2.2 → 1.4)



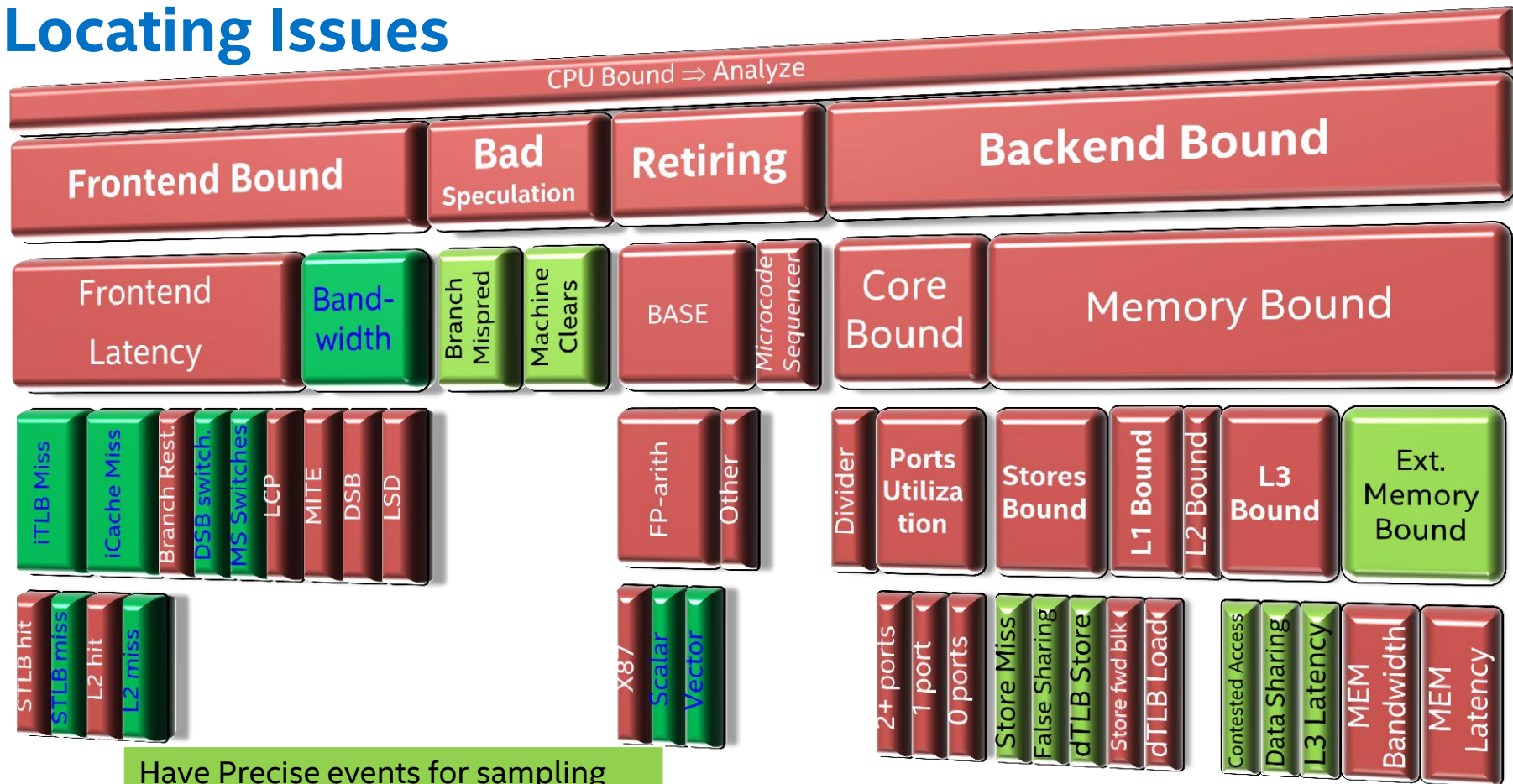
Memory Bound greatly increased



L3 Cache Bound → External MEM Bound

TMAM identifies true bottlenecks for multi-core workloads as well

Locating Issues



Have Precise events for sampling

Precise events added in Skylake

Agenda

- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

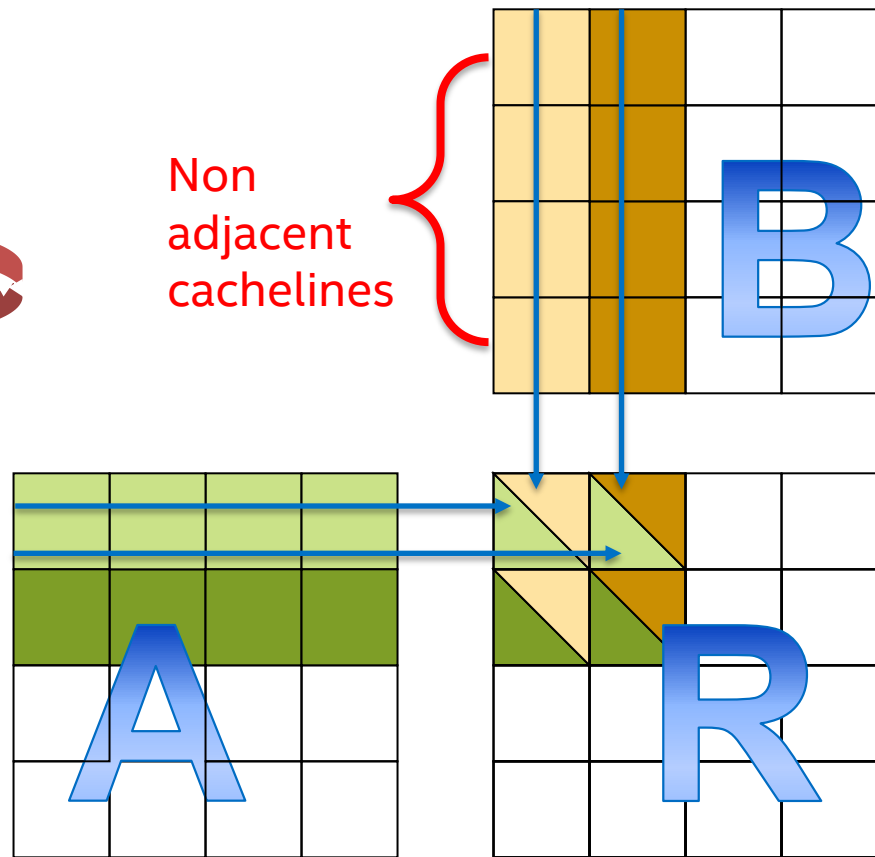
Move to Demo

Example: Matrix Multiply



Matrix Multiply insight

```
for (int i=0; i < rows; i++)  
  for (int j=0; j < cols; j++)  
    for (int k=0 ; k < cols; k++)  
      R[i][j] = R[i][j] +  
        A[i][k] * B[k][j];
```

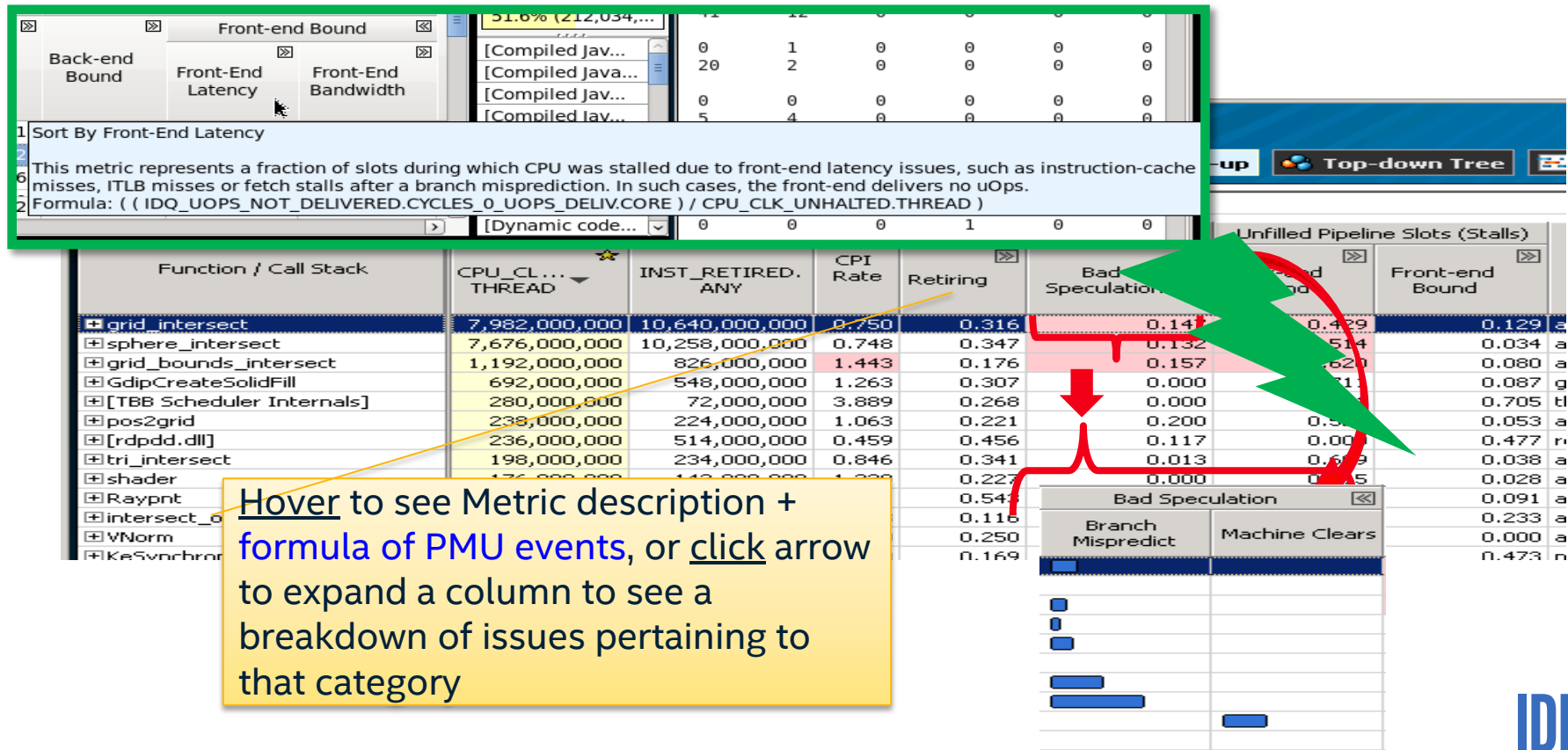


Fix memory access pattern (Loop Interchange optimization)



DEMO

VTune's New General Exploration Interface



Analysis is made easier -- "Follow the Pink"

General Exploration - Sandy Bridge / Ivy Bridge 0 General Exploration viewpoint (change)

Analysis Target Analysis Type Summary Bottom-up Top-down Tree Tasks and Frames

Grouping: Function / Call Stack

Function / Call Stack

Unfilled Pipeline Slots (Stalls)

Back-end Bound

Memory Bound

L1 Bound

	Loads Blocked by Store Forwarding	Split Loads	4K Aliasing	DTLB Overh ...	L2 Bound	L3 Bound	DRAM Bound	Store Bound	Core Bound	Front-end Bound
[Loop at line 201 in compute_bodies]	0.000				0.000	0.000	0.000	0.000	0.107	0.016
[Loop at line 234 in compute_bodies]	0.000				0.000	0.001	0.000	0.004	0.140	0.079
[Loop at line 160 in compute_bodies]	0.000				0.000	0.000	0.000	0.000	0.000	0.020
Clsqrt	0.000				0.000	0.000	0.000	0.000	0.065	0.033
math_exit	0.002				0.002	0.000	0.000	0.000	0.027	0.042
Selected 1 row(s):	0.001				0.000	0.000	0.000	0.000	0.107	0.016

Here is the problem, read tooltip to learn more

Expand each column marked pink until you come to the actual issue

Loads are blocked during store forwarding for a significant proportion of cycles. Use source/assembly view to identify the blocked loads, then identify the problematically-forwarded stores, which will typically be within the ten dynamic instructions prior to the load. If the forwarding store is smaller than the load, change the store to be the same size as the load.

Threshold: $(((13 * LD_BLOCKS.STORE_FORWARD) / CPU_CLK_UNHALTED.THREAD) > 0.05) * (CPU_CLK_UNHALTED.THREAD / > 0.05)$

Pick the actual hardware event from the formula
(LD_BLOCKS.STORE_FORWARD – typically counts bigger-size loads blocked by smaller stores to the same address) for further detailed analysis

Ack: Stas Bratanov

Un-tuned

General Exploration General Exploration viewpoint (change) ?

Analysis Target Analysis Type Collection Log Summary Bottom-up Top-down Tree Tasks and Frames

Grouping: Function / Call Stack

Function / Call Stack	Hardware Event C...	Hardware Event...	Filled Pipeline Slots		Unfilled Pipeline Slots (Stalls)		
	CPU_CLK_U... THREAD	INST_RETIRED ANY	CPI Rate	Retiring	Bad Speculation	Back-end Bound	Front-end Bound
multiply1	488,292,732,438	43,100,064,650	11.329	0.022	0.001	0.974	0.003
KeWaitForMultipleObjects	86,000,129	14,000,021	6.143	0.081	0.244	0.430	0.244
KeSetTimer	86,000,129	6,000,009	14.333	0.000	0.000	0.919	0.081

General Exploration

General Exploration viewpoint (change)

Analysis Target

Analysis Type

Collection Log

Summary

Bottom-up

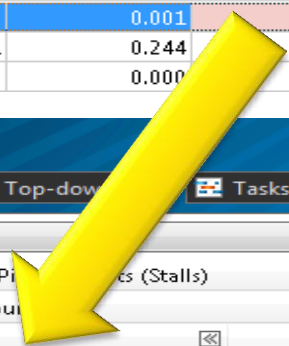
Top-down

Tasks and Frames

mult

Grouping:
Function / Call Stack

Function / Call Stack	Filled Pipeline Slots		Unfilled Pipeline Slots (Stalls)							
	Retiring	Bad Speculation	Back-end Bound					Core Bound	Front-end Bound	
			L1 Bound	L2 Bound	L3 Bound	DRAM Bound	Store Bound			
multiply1	0.022	0.001	0.070	0.023	0.064	0.745	0.036	0.022	0.003	
KeWaitForMultipleObjects	0.081	0.244	0.000	0.326	0.000	0.000	0.000	0.000	0.244	



Loop Interchange

General Exploration General Exploration viewpoint (change) ?

< Analysis Target Analysis Type Collection Log Summary Bottom-up Top-down Tree

Grouping: Function / Call Stack

Function / Call Stack	Hardware Ev...	Hardware Ev...	CPI Rate	Retiring	Bad Speculation	Back-end Bound	Front-end Bound
multiply2	43,980,065,970	51,604,077,406	0.852	0.353	0.001	0.573	0.073
KeSetTimer	24,000,036	0	0.000	0.000	0.000	0.000	0.000
init_arr	20,000,030	16,000,024	1.250	0.000	0.000	0.000	0.000
KeSynchronizeExecution	18,000,027	0	0.000	0.389	0.000	0.000	0.000
ExReleaseRunDownProte	14,000,021	6,000,009	2.333	0.000	0.000	0.000	0.000

General Exploration General Exploration viewpoint (change) ?

< Analysis Target Analysis Type Collection Log Summary Bottom-up Top-down Tree and Frames

Grouping: Function / Call Stack

Function / Call Stack	Memory Bound						Core Bound			
	L1 Bo.	L2 Bou...	L3 Bo.	DRA. Bou...	St. Bo.	DIV Active	Port Utilization			
							Cycles of 0 Ports Utilized	Cycles of 1 Port Utilized	Cycles of 2 Ports Utilized	Cycles of 3+ Ports Utilized
multiply2	0.060	0.000	0.000	0.066	0.137	0.000	0.133	0.353	0.324	0.200
KeSetTimer	0.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
init_arr	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
KeSynchronizeExecution	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.000	0.000	0.000
ExReleaseRunDownProte	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Selected 1 row(s):	0.060	0.000	0.000	0.066	0.137	0.000	0.133	0.353	0.324	0.200

Vectorization

General Exp... Analysis Tar...

Grouping: Function /

Selected 1 row(s):

Function / Call Stack	CPU_CLK... THREAD	INST_RETIRE... ANY	CPI Rate	Retiring	Bad Speculation	Back-end Bound	Front-end Bound
multiply2	43,980,065,970	51,604,077,406	0.852	0.353	0.001	0.573	0.073
KeSetTimer	...	...	...	...	...	...	...
init_arr	...	...	...	...	...	...	...
KeSynchronizeExecution	...	...	...	...	...	...	...
ExReleaseRunDownProte...	...	...	...	...	...	...	...

Hardware Eve... are Eve...

Function / Call Stack

CPU_CLK...
THREAD

INST_RETIRE...
ANY

CPI Rate

Retiring

Bad Speculation

Filled Pipeline Slots

Unfilled Pipeline Slots (Stalls)

Back-end Bound

Memory Bound

L1 Bo. L2 Bou.. L3 Bo. DRAM Bound Store Bound Core Bound Front-end Bound

multiply2	30,810,046,215	24,828,037,242	1.241	0.225	0.001	0.042	0.047	0.010	0.213	0.167	0.172	0.020
ExfAcquirePushLockExclusive	32,000,048	16,000,024	2.000	0.000	0.656	1.000	0.000	0.000	0.000	0.000	0.750	0.000
KeSynchronizeExecution	30,000,045	4,000,006	7.500	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
init_arr	28,000,042	24,000,036	1.167	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
KeStackAttachProcess	16,000,024	6,000,009	2.667	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Selected 1 row(s):	30,810,046,215	24,828,037,242	1.241	0.225	0.001	0.042	0.047	0.010	0.213	0.167	0.172	0.020

FMA (Fused Multiply Add)

General Exploration General Exploration viewpoint (change) Intel VTune Amplifier XE 2016

Collection Log Analysis Target Analysis Type Summary Bottom-up PMU Events Platform

Grouping: Function / Call Stack

Function / Call Stack	Clockticks	Instructions Retired	CPI Rate	Front-End Bound	Bad Speculation	Back-End Bound		Retiring
						Memory Bound	Core Bound	
multiply3	15,678,023,517	24,822,037,233	0.632	0.001	0.001	0.318	0.301	0.379

Function / Call Stack	Clockticks	Instructions Retired	CPI Rate	Front-End Bound	Bad Speculation	Back-End Bound		Retiring
						Memory Bound	Core Bound	
multiply4	10,696,016,044	10,332,015,498	1.035	0.015	0.000	0.511	0.266	0.208
func@0x14015ae00	40,000,060	4,000,006	10.000	0.000	0.000	0.000	1.000	0.000
init_arr	16,000,024	34,000,051	0.471	0.000	0.000	0.000	1.000	0.000
func@0x1400c0900	12,000,018	2,000,003	6.000	0.000	0.000	0.000	1.000	0.000

Matrix Multiply Summary

A function is *iteratively* analyzed

- 1 Big matrices in memory
 - multiply1 is **Ext. Memory Bound**
- 2 Loop Interchange
 - multiply2 becomes **Core Bound** due to execution ports high utilization
- 3 Vectorization
 - multiply3 reduces ports pressure but workload is still **Core Bound** and **Memory Bound**
- 4 FMA – Fused Multiply-Add
 - multiply4 further reduces instruction count with better latency/throughput in Skylake. Workload back to **Memory Bound**

step	Optimization	Time [s]	speedup
1	Textbook (baseline)	41.3	1.0x
2	Loop Interchange	5.1	8.1x
3	+Vectorization	4.1	10.1x
4	+FMA	2.8	14.7x

Performance bottlenecks vary as the code is optimized

Agenda

- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

Intel® Core™ Processor PMU Refresher

- Architectural PMU (Performance Monitoring Unit)
 - 3 fixed counters
 - Instructions Retired, Core- and Reference-Unhalted-Cycles
 - 4 general-purpose counters
 - Expands to 8 when SMT is off
 - Global Status and Controls
- Rich list of performance events
 - Most useful events are model-specific
 - A subset of 7 events are architected
- Advanced mechanisms
 - PEBS: Precisely tags performance events & profiling data into software location
 - LBR: Non-intrusive branch recording facility

Skylake PMU Overview

- Architectural PerfMon v4
- Enriched event-list
 - **Enhancing TMAM** with more top-down oriented events
 - Precise Front-end Events
- PEBS Enhancements
 - Included TimeStamp Counter (TSC)
 - Fixed multi-counter PEBS x-talk issue
- LBR Enhancements
- Intel Processor Trace v2 facilitates responsiveness analysis

Feature		Haswell	Skylake
Arch PerfMon	Version	3	4
	VMM/OS can virtualize Global Status	No	Yes
	Robust Interface for freeze on PMI		Better/faster
	In-use register		Architectural
	Secure mode	n/a	Yes (w/ SGX)
PEBS	Frontend Events	No	Yes
	TSC in record	No	Yes
	Multi counter indication	Inaccurate	Accurate
LBR	Timing info	No	Yes
	# entries	16	32
	# MSRs / entry	2	3

PEBS – Precise Events Based Sampling. LBR – Last Branch Records

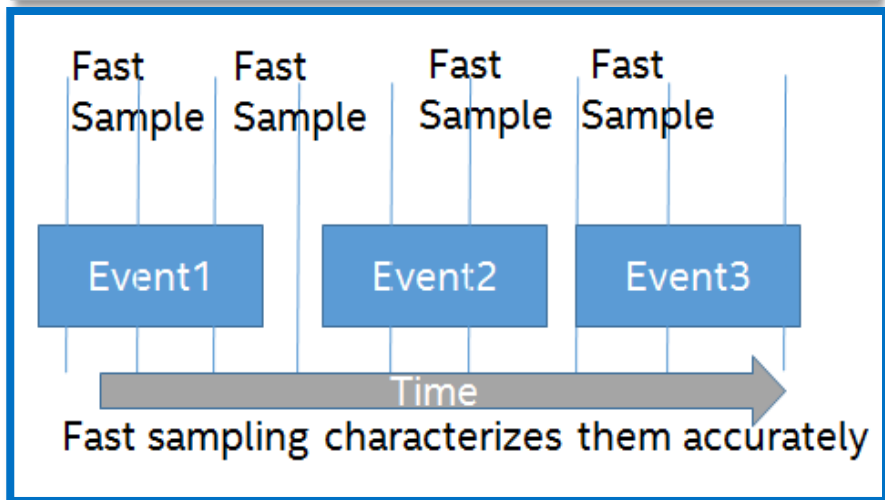
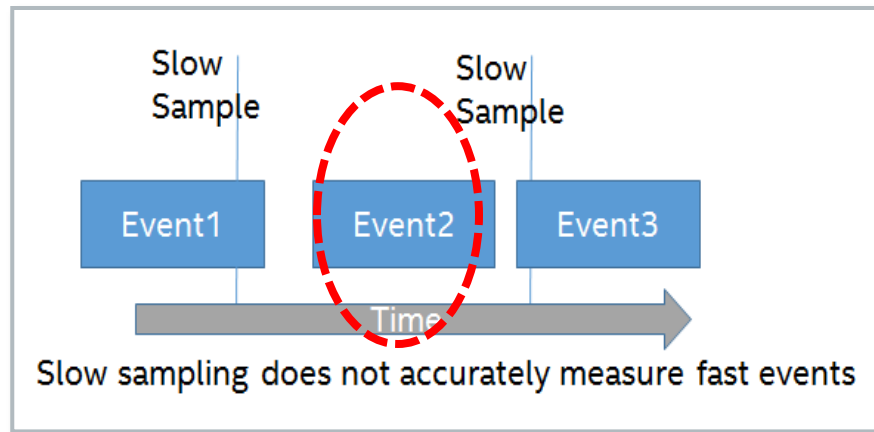
Precise Front-end Events

- Precise Event-Based Sampling (PEBS)
 - Reports precise information like Instruction address of cache misses
- Skylake expands PEBS to support Front-end events
 - These are the hardest to locate in code since they are the farthest from retirement
 - Adding at-retirement version of five key miss events in addition to a generic event

Event	Sub-Event	Description
FRONTEND_RETIRED	L1I_MISS	Retired Instructions following an i-cache L1 miss
	L2_MISS	Retired Instructions following an i-cache L2 miss
	ITLB_MISS	Retired Instructions following an iTLB miss (1 st level)
	STLB_MISS	Retired Instructions following an iTLB & STLB miss (2 nd level)
	DSB_MISS	Retired Instructions following a DSB (uop cache) miss
	IDQ_READ_BUBBLES	Retired Instructions following a “certain” IDQ read bubbles pattern (configurable latency/BW conditions). Example: periods with uop delivery starvation $\geq x$ cycles

PEBS Enhancements

- Skylake PEBS features improve interrupt less sampling
 - Timestamp (TSC) with each record
 - Applicable Counters bit-field
- Newly Enabled usages
 - Low overhead profiling
 - Interrupt less sampling allows fast sampling at reasonable overhead
 - Profiled regions with masked interrupts
 - Multi record PEBS buffering for system-wide profiling



Accurate basic-block timing

- LBRs - Last Branch Records
 - A buffer of recently taken branches
- Skylake Enhancements
 - **Timed LBRs**
 - **Accurate # of cycles between taken branches**
 - **Key enabler for new usages – examples:**
 - Time function calls; loop iterations,
 - Min/max/avg time per basic-block,
 - Potential speedup when using a perf simulator →
 - **2x Buffer size**
 - Skylake doubled number of entries from 16 up to 32

Today			
Address ▲	Source Line	Assembly	Hardware Event Count CPU_CLK_UNHALTED...
0x401005		Block 1:	
0x401005		jmp 0x411950 <Block 2>	7,404,306
0x411950		Block 2:	
0x411950	1	push ebp	10,266,120
0x411951	1	mov ebp, esp	177,420
0x411953	1	push ecx	
0x411954	3	mov eax, dword ptr [ebp+0x8]	10,058,619
0x411957	3	add eax, dword ptr [ebp+0xc]	5,094,074
0x41195a	3	mov dword ptr [ebp-0x4], eax	5,500,459
0x41195d	4	mov esp, ebp	14,726,529
0x41195f	4	pop ebp	2,468,698
0x411960	4	ret	131,662

Potential usage

Function / Call Stack	Hardware Event Count CPU_CLK_UNHALTED,...	CPI Rate	Ideal CPI	Potential Speedup
main	166,948,980	0.427	0.262	37%
multiply	61,921,921	0.421	0.400	5%
add	55,827,887	0.408	0.286	30%

Agenda

- Performance Analysis & Challenges
- Top-down Microarchitecture Analysis Method
- Demo using Intel® VTune™ Amplifier XE
- Skylake Performance Monitoring Unit Enhancements
- Summary and Q/A

Summary

- TMAM: Top-down μ arch Analysis Method

- Simplifies Performance Analysis using a structured hierarchy

- Eliminates guess work
- Reduces microarchitecture high learning curve

- Forward compatibility, consistency across IA processors

- Skylake PMU adds key capabilities

- Precise Front-end events
- Better Sampling
- Fine granular timing



Additional Sources of Information

- A PDF of this presentation is available from our Technical Session Catalog: www.intel.com/idfsessionsSF. This URL is also printed on the top of Session Agenda Pages in the Pocket Guide.
- Technical info
 - Intel® Architecture [Optimization Reference Manual](#), Top-Down Analysis Method chapter
 - A Top-Down Method for Performance Analysis and Counters Architecture, Ahmad Yasin. *In IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2014*. [[paper](#)] [[slides](#)]
- More web based info:
 - <https://software.intel.com/en-us/intel-vtune-amplifier-xe/try-buy> (a free 30 day evaluation VTune)
 - <https://github.com/andikleen/pmu-tools/wiki/toplev-manual> (Linux perf wrapper by Andi Kleen)
 - Native Linux perf support for TMAM-Level1 is coming soon.
 - Developers: check out TMAM-metric files: <https://download.01.org/perfmon/>

Other Technical Sessions

Session ID	Title	Day	Time	Room
✓ ARCS001	Intel® Architecture, Code Name Skylake Deep Dive: A New Architecture to Manage Power Performance and Energy Efficiency	Tues	1:15	2006
✓ ARCS002	Software Optimizations Become Simple with Top-Down Analysis Methodology on Intel® Microarchitecture, Code Name Skylake	Tues	2:30	2006
ARCS003	Intel® Architecture Code Name Skylake Deep Dive: Hardware-Based Security for Windows® 10	Tues	4:00	2006
Special	Zoom-in on Your Code with Intel® Processor Trace and Supporting Tools	Tues	5:30	Showcase Networking Plaza

✓ = DONE

Completing an Online Session Evaluation by 10am Tomorrow Automatically Enters You in a Drawing to Win!

You will receive an email with a link to the online evaluation prior to the end of this session.

Day 1 Prize

Intel® Compute Stick (20)



Day 2 Prize

Microsoft* Surface* 3 (6)



Day 3 Prize

Dell Venue 10 7000 Series (4)



Winners will be notified by email

Copies of the complete sweepstakes rules are available at the Info Desk

Q&A

ahmad.yasin@intel.com

The background is a solid blue gradient. In the top-left and bottom-right corners, there are decorative elements consisting of overlapping squares and circles, some of which are filled with a grid pattern. The text is centered in the middle of the image.

WHAT WILL YOU DEVELOP?

Legal Notices and Disclaimers

Intel technologies' features and benefits depend on system configuration and may require enabled hardware, software or service activation. Learn more at intel.com, or from the OEM or retailer.

No computer system can be absolutely secure.

Tests document performance of components on a particular test, in specific systems. Differences in hardware, software, or configuration will affect actual performance. Consult other sources of information to evaluate performance as you consider your purchase. For more complete information about performance and benchmark results, visit <http://www.intel.com/performance>.

Cost reduction scenarios described are intended as examples of how a given Intel-based product, in the specified circumstances and configurations, may affect future costs and provide cost savings. Circumstances will vary. Intel does not guarantee any costs or cost reduction.

This document contains information on products, services and/or processes in development. All information provided here is subject to change without notice. Contact your Intel representative to obtain the latest forecast, schedule, specifications and roadmaps.

Statements in this document that refer to Intel's plans and expectations for the quarter, the year, and the future, are forward-looking statements that involve a number of risks and uncertainties. A detailed discussion of the factors that could affect Intel's results and plans is included in Intel's SEC filings, including the annual report on Form 10-K.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

Intel does not control or audit third-party benchmark data or the web sites referenced in this document. You should visit the referenced web site and confirm whether referenced data are accurate.

Intel, Core, VTune, and the Intel logo are trademarks of Intel Corporation in the United States and other countries.

*Other names and brands may be claimed as the property of others.

© 2015 Intel Corporation.

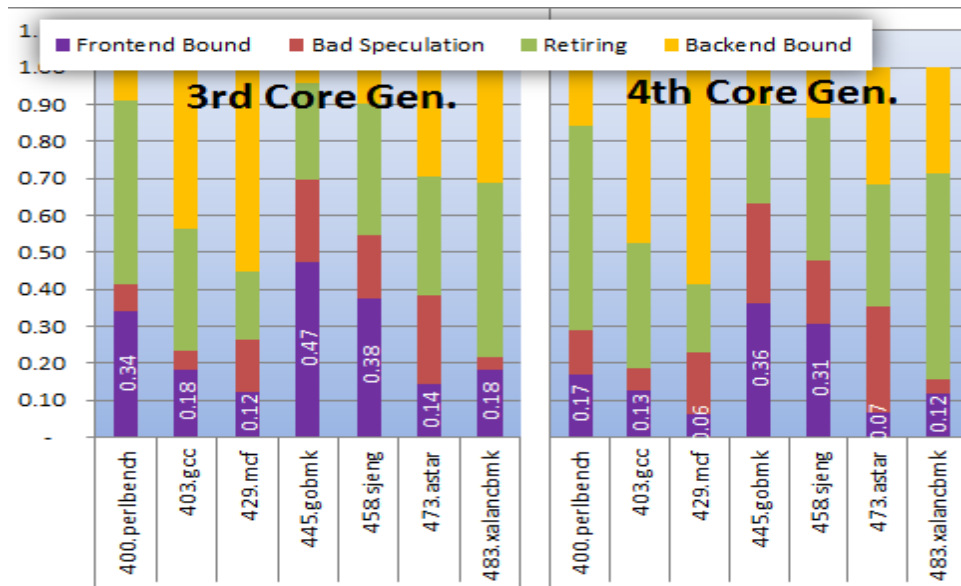
Risk Factors

The above statements and any others in this document that refer to plans and expectations for the second quarter, the year and the future are forward-looking statements that involve a number of risks and uncertainties. Words such as "anticipates," "expects," "intends," "plans," "believes," "seeks," "estimates," "may," "will," "should" and their variations identify forward-looking statements. Statements that refer to or are based on projections, uncertain events or assumptions also identify forward-looking statements. Many factors could affect Intel's actual results, and variances from Intel's current expectations regarding such factors could cause actual results to differ materially from those expressed in these forward-looking statements. Intel presently considers the following to be important factors that could cause actual results to differ materially from the company's expectations. Demand for Intel's products is highly variable and could differ from expectations due to factors including changes in business and economic conditions; consumer confidence or income levels; the introduction, availability and market acceptance of Intel's products, products used together with Intel products and competitors' products; competitive and pricing pressures, including actions taken by competitors; supply constraints and other disruptions affecting customers; changes in customer order patterns including order cancellations; and changes in the level of inventory at customers. Intel's gross margin percentage could vary significantly from expectations based on capacity utilization; variations in inventory valuation, including variations related to the timing of qualifying products for sale; changes in revenue levels; segment product mix; the timing and execution of the manufacturing ramp and associated costs; excess or obsolete inventory; changes in unit costs; defects or disruptions in the supply of materials or resources; and product manufacturing quality/yields. Variations in gross margin may also be caused by the timing of Intel product introductions and related expenses, including marketing expenses, and Intel's ability to respond quickly to technological developments and to introduce new products or incorporate new features into existing products, which may result in restructuring and asset impairment charges. Intel's results could be affected by adverse economic, social, political and physical/infrastructure conditions in countries where Intel, its customers or its suppliers operate, including military conflict and other security risks, natural disasters, infrastructure disruptions, health concerns and fluctuations in currency exchange rates. Results may also be affected by the formal or informal imposition by countries of new or revised export and/or import and doing-business regulations, which could be changed without prior notice. Intel operates in highly competitive industries and its operations have high costs that are either fixed or difficult to reduce in the short term. The amount, timing and execution of Intel's stock repurchase program could be affected by changes in Intel's priorities for the use of cash, such as operational spending, capital spending, acquisitions, and as a result of changes to Intel's cash flows or changes in tax laws. Product defects or errata (deviations from published specifications) may adversely impact our expenses, revenues and reputation. Intel's results could be affected by litigation or regulatory matters involving intellectual property, stockholder, consumer, antitrust, disclosure and other issues. An unfavorable ruling could include monetary damages or an injunction prohibiting Intel from manufacturing or selling one or more products, precluding particular business practices, impacting Intel's ability to design its products, or requiring other remedies such as compulsory licensing of intellectual property. Intel's results may be affected by the timing of closing of acquisitions, divestitures and other significant transactions. A detailed discussion of these and other factors that could affect Intel's results is included in Intel's SEC filings, including the company's most recent reports on Form 10-Q, Form 10-K and earnings release.

Backup

Microarchitecture Comparison

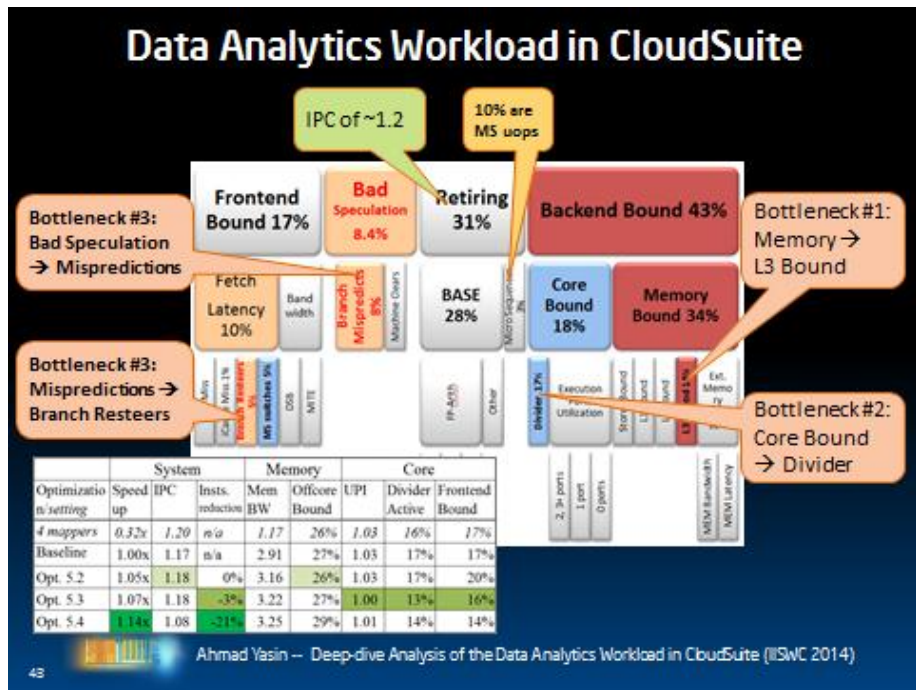
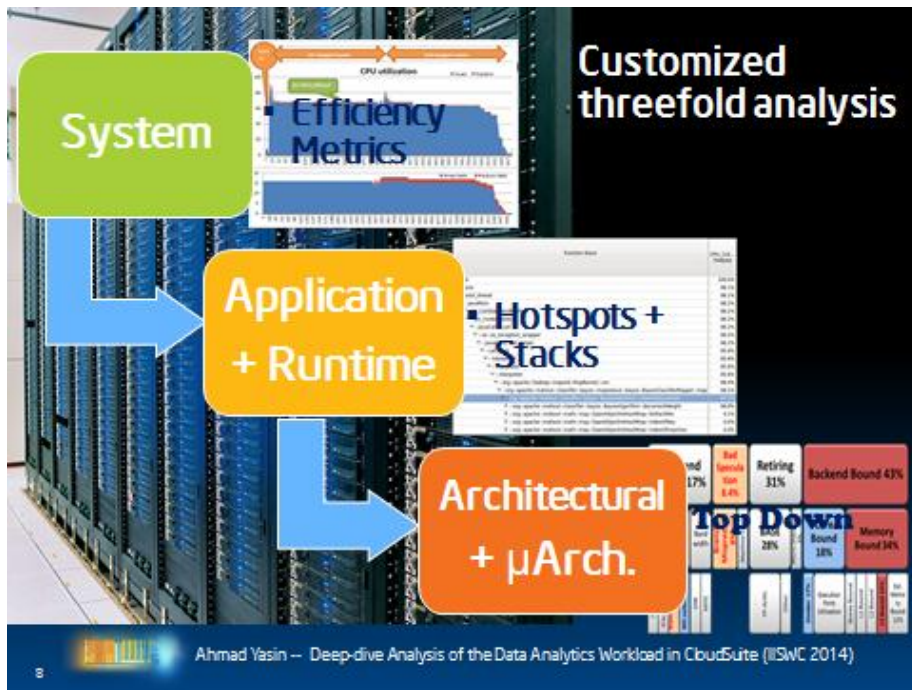
- Haswell (4th generation Intel® Core™ processor) has improved front-end
 - Speculative i-TLB and i-cache accesses with better timing to improve the benefits of prefetching
- Benefiting benchmarks clearly show reduction in Frontend Bound



Using TMAM, forward compatibility is assured on Intel Core

Sample Workload Optimization

Deep-dive Analysis of the Data Analytics Workload in CloudSuite - Ahmad Yasin, Yosi Ben-Asher, Avi Mendelson. In *IEEE International Symposium on Workload Characterization, IISWC 2014*. [[paper](#)] [[slides](#)]



Datacenter Profiling

- Profiling a Warehouse-Scale Computer - S. Kanev, J. P. Darago, K. Hazelwood, P. Ranganathan, T. Moseley, G. Wei and D. Brooks, in International Symposium on Computer Architecture (ISCA), June 2015.
- First to *profile* a production datacenter
 - Mixture of μ -arch bottlenecks
 - Stalled on data most often
 - Heavy pressure on i-cache
 - Compute in bursts
 - Low memory BW utilization

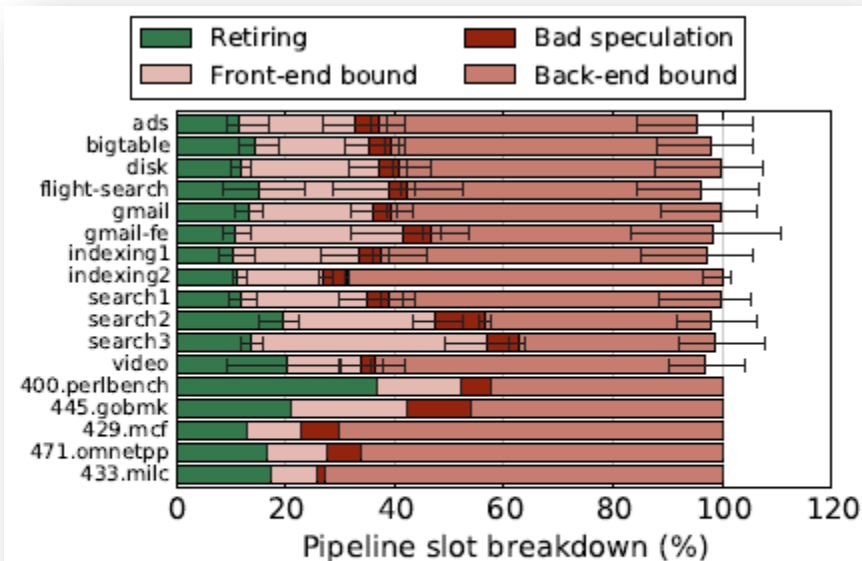


Figure 4: Top-level bottleneck breakdown. SPEC CPU2006 benchmarks do not exhibit the combination of low retirement rates and high front-end boundedness of WSC ones.

Arch PerfMon Version 4

- Better support to *virtualize & share* the Architectural PMU
 - IA32_PERF_GLOBAL_STATUS (0x38E) had per-counter overflow status bit
 - New IA32_PERF_GLOBAL_STATUS_SET enables setting bits in 0x38E
 - Enables a VMM to virtualize overflow status bits (only)
 - New Counters Frozen interface
 - New LBRs Frozen interface
 - In addition, IA32_PERF_GLOBAL_INUSE facilitates sharing the PMU among multiple agents (Architectural register at address 0x392)

IA32_PERF_GLOBAL_STATUS (0x38E)

63	62	61	60	59	58	57:35	34	33	32	31:8	7	6	5	4	3	2	1	0
CondChg	BufferOvf	UnCore Ovf	interfer ence*	Counters Frozen	LBRs Frozen	Rsvd	OvrFxCtr 2	OvrFxCtr 1	OvrFxCtr 0	Rsvd	OvrCtr 7	OvrCtr 6	OvrCtr 5	OvrCtr 4	OvrCtr 3	OvrCtr 2	OvrCtr 1	OvrCtr 0
0	0	0	0	0	0	0 (GP)	0	0	0	0 (GP)	0 (wr '1' #GP if HT on)				0	0	0	0